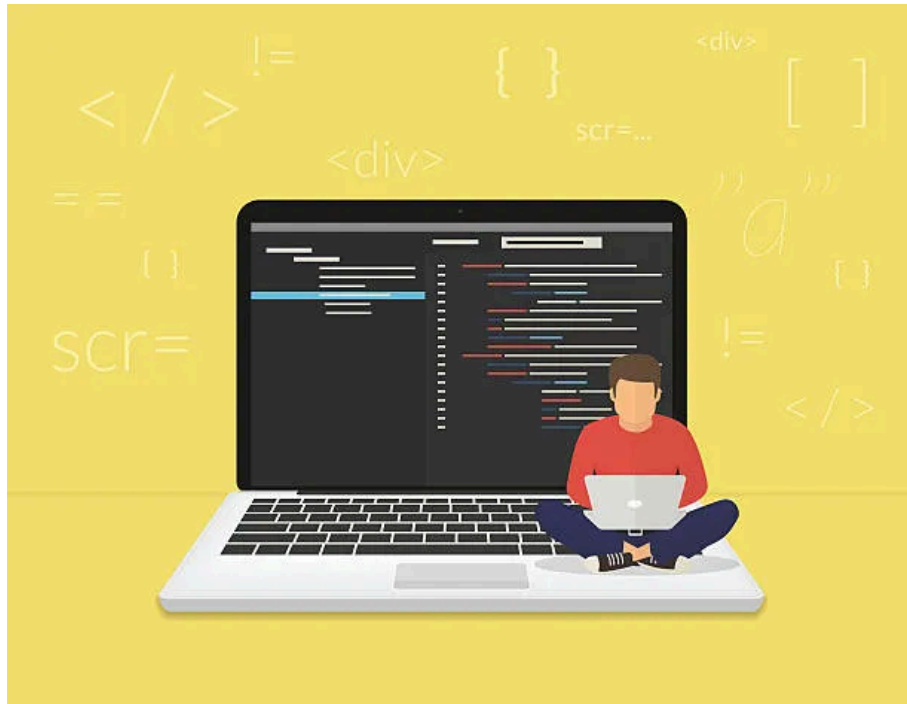


# Deliverable 2: Documentation



## Autors

Pol Ortega Fernàndez: 49254483Y

Andreu Puig-gròs Munté: 48255353L

Marc Pujol Navajo: 49259109D

## Escola Politècnica Superior

**Work presented to:** Roberto Garcia and David Sarrat

**Subject:** Web project

**Course:** 2025-2026

**Date of delivery:** 15 of May, 2026

## Contents

|                                                                                    |          |
|------------------------------------------------------------------------------------|----------|
| <b>Introduction.....</b>                                                           | <b>1</b> |
| <b>1. Github public address.....</b>                                               | <b>2</b> |
| 1.1. Project Source Code & Open Source Repository.....                             | 2        |
| 1.2. Accessing the Repository.....                                                 | 2        |
| 1.3. What can be found in the repository.....                                      | 2        |
| 1.4. Users.....                                                                    | 2        |
| <b>2. Notes about all the design decisions.....</b>                                | <b>3</b> |
| 2.1. Database updates.....                                                         | 3        |
| 2.2. Implementation of matches on the server side.....                             | 6        |
| 2.3. Reviews system.....                                                           | 7        |
| 2.4. Web pages to display and allow browsing application data.....                 | 8        |
| 2.5. End-to-end tests for the corresponding features of the application (BDD)..... | 9        |
| 2.6. API to incorporate data in the operation of the application.....              | 11       |

## Introduction

The following report provides a comprehensive yet simple technical breakdown of the development process for the Django-based web application presented in this subject. It's designed to facilitate the access and data management of a competitive Counter-Strike 1.6 related website that is exclusively focused on 1v1 game matches inside the web browser along ranking and user customization features.

This project was developed as a direct evolution of the initial proposal made by the professors' team. It incorporates the given constructive feedback and technical requirements to ensure a robust final product. This second deliverable includes a reviews system in which players can publish their opinion. We've almost finished the implementation needed to allow players to play matches against each other and gather the log data from the matches in order to store it in the web database, and modify users' parameters (we still have to solve some matchmaking issues in our own code implementation or find a better API), and finish joining the database processes with the game server's life cycle). In addition, the database has been filled with some fixture data of countries (obtained from a new API) and maps (the approved ones for the project) and some functions (linked to the matching functionality and table dependencies) to prepare the website for the game hosting features that will be in the last deliverable.

In order to ensure the high standards of software portability and scalability, the app is containerized using Docker and orchestrated via Docker Compose. just like the previous deliverable. Furthermore, it adds some end-to-end tests for the main features of the application. The main ones are focused on the review system because it is the main feature requested for this deliverable (testing that everything behaves as expected in every single test). It also uses the data from the database we've defined in the project in order to assist the users in finding other web players to leave, modify or delete reviews.

This documentation provides an overview of our proposed solution, and it includes the revised data model documentation, along with our database models' implementation design, some of the design decisions, and the detailed instructions on how to run/deploy our application and use the users from it.

# 1. Github public address

## 1.1. Project Source Code & Open Source Repository

The development of the Strike 1v1 project is publicly accessible on the GitHub platform. By doing it this way, we can allow everyone to see the main web page's source code as well as, once finished, the implementation of other features such as matchmaking and the ability to play through the website. Also, by hosting our source code on GitHub, we can also let anyone interested review our multiple open issues, pull requests, and all the program logic. They will also be able to track the whole evolution of the platform from its alpha stages (preliminary) to the production environment (advanced).

## 1.2. Accessing the Repository

The GitHub public address for the project's source code is the following one:

- **GitHub URL:** <https://github.com/MarcPujolUNI/Strike1v1>

Also, we provide the link to the Strike 1v1 server, where we are developing the server to play:

- **GitHub URL:** <https://github.com/Apm-96/1v1Strike-server>

## 1.3. What can be found in the repository

The core engine of our website has been implemented using the Django framework, obviously with its latest version, 6.x. All the frontend and backend elements in our project are based on this framework. The frontend assets will let you see the Tailwind CSS configuration, which allowed the development of the website's main design features. The design is based on a retro style with some added modern elements that cause the website to have the typical look that these kinds of platforms normally have.

Furthermore, all the database migrations are also available. Those are the main database tables that the platform will use to ensure that all the features the project wants to achieve are indeed completed.

## 1.4. Users

This project, obviously, also has an admin account, which is accessible with the following information:

- **User:** admin (we thought that naming it this way would allow it to be easily recognizable).
- **Password:** admin (we thought that giving it the same password would help remember it).

We've also added other users to test the application from the point of view of a normal user (below):

- **Users:** marc, andreu and pol, which refer to the names of the Strike 1v1 project's members.
- **Passwords:** programari, telecomunicacions and computació, in the same order as the users.

## 2. Notes about all the design decisions.

In the following section, we will visit each one of the main points related to the new features that we've added, and we will also see why all of those design decisions were made, in this second deliverable.

### 2.1. Database updates.

First of all, we did some updates in our backend structure: in other words, the database. Basically, we added some new functions in the `models.py` file (and outside of it as well), so as to better handle the new features that we'll be adding in the future, along with the necessary database changes. We'll now go through every database relation and explain what was changed either directly or indirectly to make things work. The `Map` relation itself hasn't really changed much: outside of simplifying the map type categories to `Big` and `Small`, since, for now, those suffice (real CS maps didn't fulfill our current criteria so we made 3 maps for the website: we might add more in the future). Apart from limiting the choices length to 10 (we don't need more for now) we also added a json file with those maps' details manually, obtained with a specific script related to the game server. We loaded it to our database using `python manage.py loaddata map_fixture.json` (the name of the file). The other fixture related relation, `Country`, saw a bit more changes, actually: apart from the default country constant changing its value, from 1 to 9 (the new pk from the Undefined country, previously known as Unknown, the one with the Antarctica's flag), and the tries for the image fetching being risen to 5 as well as no longer deleting the instance if the image can't be fetched, we changed the flag API. That's because the previous API didn't have all the images, and some fields were incomplete. The 2 new APIs, apart from giving the images in way more formats, give us endpoints that provide countries' lists, ISO codes, and other country attributes.

The country fixtures were obtained via a python script, `generate_fixtures.py`, that fetches all the data we need (country name and ISO name (the `cca2` type)) and sorts it before building the json entries with that info (using `enumerate` for the pk, and paying attention to the Undefined's special case). The logic for the flag image fetching has also been moved: it's now a `manage.py` command, as the fixtures didn't automatically call `save` and therefore didn't fetch the image like we wanted. In order to prepare for other possible situations, we defined some extra options for that `populate` command (`fetch images` for specific countries, and `replace all images`). It iterates through the chosen countries, calls the API to get those, deletes the previous ones and updates both the memory, and the image field. Then, we have the `WebUsers`, the main table in the first deliverable. Just like email in the previous one, for the same reason (validators), we overrode it to define a concrete length range ([3-50]). Apart from other minor changes, we added several important functions. The first one is a function used to automate the corresponding `CounterUsers`' creation when a `WebUser` is created, and it's done via `save`. If that user is being inserted (we had trouble with the `pk is None` conditional), we create its new `CounterUser`: the good news is we only need the `WebUser` to do so since everything else starts being zero. As any user's deletion triggers the `on delete Cascade`, we now have those classes tightly coupled, which was truly our goal: `CounterUsers` can't exist without `WebUsers` since it's just a way of storing its info separately.

We also fetch the old country and old username before saving, because, unlike the rest of the user's parameters, their change affects other relations. If we change the country, we will also need to change the ones stored in the rankings, actually. By using the reverse relationships we have defined, we can obtain the user's entry in the ranking, as they're built on One to One relationships. We also use this to handle country deletion which we'll comment later on. After updating the countries, we now only need to update the local rankings since moving players from one country to another changes some of those ranking positions (we take both countries into account). And, as for the username, we need to update the name copies from the reviews which we keep to preserve information even after user deletion. We also need to handle the matches' copies (and stats), and we distinguish between winners and losers.

We'll continue once we move on to the Match table. The Reviews one goes first, though (since we're using update, which doesn't trigger any save logic, we don't need to add anything else to reviews for that functionality). Outside of small changes (in the parameters' definitions), nothing is touched, so we can move on to the CounterUser table now. What has been changed here is mostly the new functions we mentioned at the beginning. We have another save override that works pretty similarly to the other user's one: using state adding to know if we're inserting new users, to know exactly when we need to add registers to the ranking relations. After saving, if so, we fetch the country and enter a transaction, as we're iterating through the whole rankings table, which could affect other django threads, busy with other ranking related functions (like updating scores after a match). We will query through the DB now to obtain the highest position in the rankings so far. We use aggregate so that we can apply the Max function (the method has this name because it's an aggregate function in SQL), and we use another function, Coalesce, to handle empty results better. We extract the max position from the query (which will be 0 if there's no users yet) and we'll add one. We'll do the same for the local ranking, but, before that, we'll apply the filter function to apply a WHEN condition from SQL, so we only iterate through the registers with the same country as us. We will now create the corresponding ranking entries, using all the data we've gathered so far (we'll use the country here). Everything needs to go in the transaction block, since we have to protect ourselves from concurrent reads (queries) and writes (object creation).

The commented section below is there just for testing (so as to trigger rankings saves after modifying the score while the matches can't be received from the game server). We also have another function, used to update all the CounterUsers' parameters, using the data from MatchStats, increasing the wins or losses according to if the player actually won or lost the match. This is the function that will trigger the ranking update once everything is fully integrated. We then have the Match relation, with actually some notable changes in the parameters: first, the score's regex validator, that now makes sure that exactly one of the players got 10 kills, the exact win condition defined in the game server. We've also added the log parameter, which stores the path we'll store the matches' log file in, received from the game server as well. We've defined another command that will be in charge of deleting the logs after 90 days have passed. It filters the matches looking for older matches to delete their logs. They occupy spaces so they need to be handled properly, even though this is not automated (yet): you need to call the function in the project shell to activate it, that's why we allow logs to be null (they'll eventually be).

Here we have the continuation of the previous update call (usernames): back then, we were updating each match accordingly (differently depending if the affected user won or lost the match), but we use a constant to do so. After saving, we need to update the related match stats, but we can directly edit the related match stats, as we only have 2 match stats per match, and only one of them contains the desired user. We'll use prefetch related in order to preemptively join both match relationships in order to run these functions a bit faster, a pattern we'll repeat in other similar functions in the database. We then have match stats, where, apart from adding some reverse relations' names, we added some new verification logic using the clean function, as it was too complex for simple database constraints. This is related to the new parameter we added, the score: we're now saving the increment in the score that the users go through after each match, which can be useful if there are problems with matches, or just for general reconstructibility. We make sure winners increase their score and losers don't, since there might be cases where their score remains the same (to avoid negative numbers). We'll trigger it every time a match stats gets saved, but only for the new object (match stats, alongside matches, are never modified once created). We can see we modify the user's stats through match stats as well. And, as for the rankings, we can see we added the country parameter in the global ranking as well, because it was too far in the join chain (3 joins). Like with the countries before, we've also changed the on delete action from Protected to Set Null, as we'll allow country deletion, if it's ever needed, for some reason.

We also allow null values in the country (temporary) and in the ranking (we need that for the updates). In order to see those functions, we'll need to go to the rankings.py file: we can see we have functions there for both Global and Local rankings, where we order the whole relation by the users' score, so it involves a join, inside a transaction to avoid concurrency problems. We need to null all the positions, though, to avoid issues with the unique constraint we defined in the first deliverable. Since everything is sorted, we can just reassign all the positions with the update function, faster as it doesn't trigger any custom save logic (we're using the bulk variant, as we pass a queryset with the specific parameter we want to take into account). It's similar for the local version, even though we have to filter the country before doing everything else. We also have some interesting functions that work with signals, used for some match deletion (once both users stop existing, it makes no sense to keep those matches, so we trigger it every time a user is deleted, to check if any of its matches becomes orphan). The other one is used to assign the default country (Undefined) for the players whose country gets deleted, which is rare, but could happen. We admit null temporarily, we then set the default country and trigger the local ranking change logic (it's easier like this as setting default directly doesn't trigger the ranking update). Finally, we'll briefly mention the changes implemented in the admin interface. Now, you can't directly create maps or countries (handled by fixtures) nor counter users or rankings (handled by our creation cascade), or edit any field without getting inside the specific instance. Time fields and rankings can't be edited (unless you trigger the update logic for the latters) and only some user data can be changed (just in case a future admin could need that feature). After adding all the countries, we can see that the admin pagination works as expected (every interface got the maximum reduced to 10 instances), and another notable change was the alteration of some css to remove the delete buttons from both the game users and the rankings, since deleting the delete permission affected the cascade deletions.

## 2.2. Implementation of matches on the server side.

Also, it has been added to the project the main workflow between the web page and the server side that manages the instances of the matches. These features allow the creation of the matches' links, where the players will be able to play against them. And when a match ends, gather the data from the log generated to update the database of the website part to modify information such as the score of the players, match history, leaderboard positions, and other statistics. They're not fully implemented yet. In the following paragraphs these features will be detailed in order to justify the design decisions.

First of all, when setting up the system, the main objective was for the web to be able to instantiate a cs match when needed and to return the ip direction of this match to be played. The best solution to communicate the web and the server was by using an API attached to the server. This API is able to receive a GET to its IP and then activate the autorun script for creating the container which contains the cs match. For design purposes the selection of different maps is random: every match is a different container, so there's less configuration to be made. When the match is created the API gets the ip address of the container and it sends it to the web, later we will talk about what the web does with this information. When the match terminates, the autorun script calls the API and sends the information about the match extracted from the logs in an easy to manipulate format: json. With the ip received, the only thing our web has to do is show it to the user, with a new "page" related to the matchmaking system (we will talk about it later). And, with the results of the match we will need to update the statistics of the involved players, so we'll call methods declared on views.py to autoupdate all the relations of the user's stats. With all of this implemented now the main purpose of the site is fulfilled, but is missing something essential. To do the matchmaking we need a new page on the site, /waiting. This is the place where the user waits in the queue to meet another player to match. Our idea was to use an API, but (<https://developers.hiveplatform.ai/en/latest/api/matchmaking-api/>) (the selected API), was pretty hard to use (for instance, some parameters needed to be configured via an internal console), so we decided to build a self-hosted matchmaking system, to give the system more flexibility (while we search for other alternatives). The first implementation was a little controversial.

The main problem is where to allocate the waiting list and the match of the 2 players: the fastest way was to save on the database both data systems. When the user clicks the matchmaking button it enters /waiting page and its profile enters the queue with his name and elo, then the matchmaking algorithm checks every x time for other players on the queue with a elo margin. When found it creates a match on the database and the web calls the server API, and when it receives the IP the /waiting updates for both users and it is shown the IP. This state of the site /waiting to show the IP will remain since the match created on the database is removed, so we take into account disconnections and problems in-match. This implementation will probably be replaced with a better one, because we consider that it makes no sense to save in the database the queue, so probably in the next deliverable will be fixed. So now we have a "working" matchmaking system and the users can enjoy the service. But, in order to make the site accessible in this system, we need to deploy it to the public.



So the best solution was a VPS, but for price increases for global crisis this option was not optimal (and we would need to migrate all the system of our local server to the VPS), so we have chosen to use tunneling, specifically from the site [playit.gg](https://playit.gg) that allow us to use a tunnel for tcp and udp at the same time for game characteristics purposes. And with the public ip provided we can use reverse-dns like duckdns.org to create a custom URL like "strike1v1.duckdns.org". The site still functions in http but it just works. Finally, comment that all the implementation of this sector of the documentation is very sketchy, not polished and not final, and can be totally changed on the final delivery. If you are interested on the server-side or backend of the project, this is saved on the following GitHub repository: <https://github.com/Apm-96/1v1Strike-server>

## **2.3. Reviews system.**

Now we are going to talk about the main feature of this deliverable, which is our new reviews system. Basically, the users will now be able to add reviews to other players from the website to share their opinion about them. Also, users can update the reviews about players and delete them if necessary.

In the following paragraphs, these new features will be detailed in order to justify the design decisions. First of all, we have a search page, where users can look up the players they want to review. There, instead of implementing a huge scroll where users have to look for the player of their choice, we've implemented an internal API of our platform with a specific endpoint, which uses AJAX to query for players and autocomplete the player card information. As the platform's user base grows, rendering thousands of players within a form becomes more and more inefficient and compromises scalability, so we opted for a different approach. This search engine uses AJAX and filters through all the user profiles dynamically, displaying responsive previews matching the input in real-time (doing contains-like queries). Once the web user selects one of the options, it shows a player card from the selected user selected to verify the correct user they want to rate and later search for them. We then have the list of reviews from the player we were interested in. Here, users can read all historical feedback left by other community members, sorted chronologically (we only allow 1 active review from every user, though). Each entry displays the reviewer's name, the date, their comment, and a rate represented by a star rating (from 1 to 5, just as we defined in the DB: only whole numbers, half stars wasn't worth it). Now, if the logged-in user hasn't rated this player yet, a dedicated submission form appears at the bottom. To make it highly engaging, there is an interactive star rating system (from 1 to 5 stars) where the stars dynamically light up or change color as the user hovers their mouse over them. The user can select their rating with a click, type their detailed feedback into a text box, and submit it instantly. If it's the reviews list of the logged-in user, he will not be able to publish a review of himself. He only will be able to read other players' reviews: he only needs to click on a specific one to be redirected to a new page where he can see in detail all the feedback given by a specific player. Finally, in order to give users complete control over their content, we implemented this review detail and management screen. This view is highly personalized and enforces strict privacy rules. If a user visits the detail page of a review they wrote themselves, the screen dynamically transforms into an editing dashboard.

They can re-read their full comment, change the text, or adjust the star rating if their opinion about the player has changed over time. But, if the user decides to remove their review entirely, they can click the delete button. To avoid frustration from accidental clicks, the interface triggers a confirmation pop-up window (*DELETE? Are you sure?*). The review is permanently removed from the system only after the user explicitly confirms it. If a user visits a review written by someone else, the editing form and delete buttons are completely hidden, ensuring that nobody can alter another player's feedback. All these views have the objective that users are able to create new instances of model entities, in this case, reviews. Moreover, allow modifying these instances created by them and deleting instances.

## **2.4. Web pages to display and allow browsing application data.**

With all the backend stuff already mentioned, we can move forward to the frontend and what the user will see when it enters the website. In this section, we'll basically describe the main pages that have been implemented in this deliverable and all the information that we try to show to our future users.

First of all, we'll mention that we have been using both Figma and Stitch to create prototypes of the website, in order to make the styling part easier for us, as well as providing a minimum responsive website and not rely entirely on the implementation of raw HTML and CSS. We've never worked with these types of files, and this is useful to learn about how they work and what their main function is in a website environment. In addition, we've been using Tailwind CSS to efficiently manage all the colors from the website, and a *main.css* file. Both files are available in the static folder inside our repository. Some interactive upgrades have been implemented to some of the existing screens. The main ones are at *signup.html* and *leaderboard.html*. While the registration and leaderboard structures were initially designed with standard inputs and long country lists, they have been upgraded with an API search using AJAX. Users no longer have to scroll through exhaustive country menus. Instead, typing a country name triggers a dropdown that instantly reveals matches to that input, automatically updating country flags and regional data. We've also incorporated some icons in our page to make it more visually appealing for our users (you can see that in the main page). Now, some new screens have been added. First of all, we have the *profile.html*. This page was previously marked as under development (W.I.P. window), this screen is now fully operational as the player's personal dashboard. From this view, authenticated users can customize account preferences, edit personal registration data, or trigger the account deletion workflow if they choose to leave the platform. In some minor way, we have the *play.html* and *footer.html*, which were some of the ones marked as under-development features, where now you can contact web support and have an initial workflow to trigger the beginning of a new match matchmaking. Additionally, we have the *matches.html*, accessible from the profile settings, providing a record of all the previous player's matches. It presents a structured, paginated timeline detailing every match played, and its final outcome (wins are highlighted in green and losses in red), and historical opponent usernames, allowing users to browse their past performances easily.

We can now focus on the main HTMLs that contain the review system feature. Firstly, we have the *review.html*. This serves as the primary entry point for the player's review system. It offers a search experience where users type the username of any other player they want to rate or read reviews. To ensure clarity of who they are looking for, selecting a user from the results dynamically populates a "Preview Card" on the screen. This one shows the player avatar, country flag, and username, so they can confirm that they have selected the correct player, before unlocking the action button to proceed.

Secondly, we have the *review\_list.html*. Once the player is selected, the user is redirected to this community reviews list. This screen acts as a timeline displaying all chronological reviews, feedback scores, and star ratings left for that specific player by other community members. If the browsing user has not yet reviewed this player, a button allows them to publish a dedicated review, complete with a star rating tool that dynamically changes as the user hovers or clicks to pick a score from 1 to 5 stars. Lastly, we have the *review\_detail.html*. This screen provides management controls for a single specific review. If a user opens a review written by someone else, the page acts as a read-only document. However, if they open a review that they have written, it transforms into an editing suite. The user can rewrite their feedback text or readjust the star rating. Additionally, to avoid users from frustrating mistakes, clicking the delete button does not immediately erase data. Instead, it triggers a warning modal dialog asking for confirmation before permanently deleting the post (it's safer this way).

## 2.5. End-to-end tests for the corresponding features of the application (BDD).

With all the main features already mentioned, we can move on to talk about the automated end-to-end (E2E) testing framework implemented to verify the correct workflow of the entire website. These tests are written following Behavior-Driven Development (BDD) principles, utilizing the Gherkin syntax (Given, When, Then) inside the *features/* folder. These are the main features that are being validated:

- **Matchmaking UI (*view\_play.feature*)**. This test verifies the readiness of the primary action interface. The test interacts with the play section, ensuring the "START PLAYING" layout renders correctly, the "FIND MATCH" interaction button is fully visible, and click routines trigger the appropriate server hooks.
- **Global and Regional Leaderboards (*view\_leaderboard.feature*)**. This test validates that leaderboard standings are displayed correctly. Scenarios check that the system pulls data from the database and properly sorts the players in descending order according to their ELO (score) points. Furthermore, it validates the country filter, confirming that selecting a country dynamically narrows down the displayed list to matching regional players while hiding others.
- **Match History Logs (*view\_matches.feature*)**. This test checks the integrity of a player's match history logs. It covers an empty-state scenario to ensure that new profiles or profiles without games show the explanatory message "No matches found in the logs." On the other hand, it validates behavior with existing data, ensuring that the list of recent matches loads correctly and properly implements the table's pagination controls.

- **Player Dashboard (*view\_dashboard.feature*)**. This test confirms the correct display of the competitive status metrics for logged-in accounts. The scenario logs in with an active user to check that the "DASHBOARD" navigation link renders correctly and that the interface accurately displays the summary of the player's competitive status and account performance.
- **Legal pages (*view\_legal\_pages.feature*)**. This test validates the integrity of the informational layouts linked across the website footer. It tests navigation to the Terms of Service, Privacy Policy, and Cookie Policy. Confirming that each layout displays its appropriate title, educational disclaimers, explains academic data boundaries, and states the details regarding session tokens.
- **User registration (*sign\_up\_user.feature*)**. This test verifies the complete registration flow for anonymous players who want to join the website. The scenario simulates completing the registration form with valid credentials (username, email, and password) and linking it to an existing country, confirming that submission successfully redirects to the login view. Additionally, it validates at the database that the new record is saved properly and that the associated matchmaking statistics profile is automatically initialized.
- **User authentication (*log\_in\_out\_user.feature*)**. This test ensures that the login and logout control for registered users function without any issues. The scenarios check that logging in with valid credentials redirects the player to the landing page, displaying their personalized dashboard. It also tests the logout routine, verifying that the user is properly unauthenticated and that the interactive buttons ("Login" and "Sign Up") reappear in the website's header.
- **Profile Management (*edit\_profile.feature*)**. This test validates the update of personal information from configuration settings. Through different scenarios, it verifies that a logged-in user can modify their basic details (such as username, email, or selecting a favorite map). Also that it can update their login password by providing both the old and new credentials and upload a new image file for their avatar. Each action requires verifying an explicit success message after saving changes.
- **Account termination (*delete\_profile.feature*)**. This test evaluates the mechanisms related to account deletion at the user's request. The scenario simulates an authenticated player triggering the deletion process on their profile, requiring the system to display a confirmation modal before proceeding. Finally, it validates that the account is permanently removed from the platform and that the user is redirected to the landing page.
- **Review navigation (*view\_review.feature*)**. This test verifies that players can browse other users' reviews to evaluate their website reputation. Additionally, it exercises some security constraints, checking that the system hides the review creation mechanisms if a player attempts to visit and write a review on their own profile.
- **Review creation (*review\_create.feature*)**. This test simulates a user submitting a review for another player. It fills out the review form with a text description and a star rating, submits it, and verifies that the newly created record appears under section "Your review."
- **Review modification (*review\_update.feature*)**. This test validates the edit workflow for an existing review. It ensures authors can access their review details, click the modification

controls, rewrite comments, adjust ratings, and save the updates. Additionally, it verifies that security barriers block non-authors from viewing or interacting with modification controls on reviews written by other users.

- **Review deletion (*review\_delete.feature*)**. This test focuses on the elimination process. It ensures a reviewer can delete their own review post. The test verifies that clicking the "Delete" button invokes a confirmation warning modal before any records are deleted from the database. It also ensures unauthorized users cannot see or delete other players' posts.

## 2.6. API to incorporate data in the operation of the application.

Now we can talk about the API implemented in the website workflow. To avoid unnecessary page reloads and improve the performance, several views incorporate communication with the website's own data via AJAX (fetch API). These endpoints serve filtered database records directly to the UI. Additionally, an external data source is utilized to manage countries' media assets (flag images). The following subsections detail the purpose and structure of both internal AJAX services and external asset countries' integrations within this deliverable. First of all, we have the implementation of Django REST Framework generic list views optimized to handle real-time queries from frontend components. It has to main implementation that we consider.

- **Web User Search API (*/api/search/*)**. This internal API is integrated into the entrypoint to the reviews system (*reviews.html*). As the web user types a username, a JavaScript function dispatches a request to this endpoint. It queries the WebUser model using case-insensitive partial matching (*icontains*). This one returns a JSON payload serialized by WebUserSerializer. This allows users to view live previews of player identity cards, including user avatars and country designations, before selecting a profile to look for on the website.
- **Country Search API (*/api/countries/*)**. This internal API is integrated across multiple interfaces such as the leaderboard page (*leaderboard.html*) and the signup form (*signup.html*). As the web user types a country name, a JavaScript function dispatches a request to this endpoint. It queries the Country model using case-insensitive partial matching (*icontains*). This one returns a JSON payload serialized by APICountrySearchList. It shows real-time autocomplete suggestions to the client, fetching country names and the flag assets.

Finally, we have the external flag API integration. This one basically is used to gather all the countries' flag images dynamically as they are inserted into the respective Country table. Basically, for every country, it makes a call to this API to gather the flag media asset and store it locally in the database.

- **FlagCDN (<https://flagcdn.com/>)**. Instead of relying on hard-coded static uploads for each nation's flag, the backend handles this dynamically inside the Country model. When a new country is registered, the overridden *save()* method triggers a network request to FlagCDN's public endpoint. It fetches high-quality PNG using the standardized ISO code (*country\_iso*).

- **Resilience and Persistence.** To ensure high availability and prevent broken images in the UI if the API is no longer available. Basically, the backend includes an automated retry loop (up to 3 attempts). Once a valid request is verified (HTTP status code 200), the image is saved directly into the local project storage through Django's ContentFile utility. This allows maintaining database consistency and faster load times for the frontend content.